

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2– Coding Challenge

Course Code:	DSA0404	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO5 –Coding Challenge	Assessment:	Assessment Tool 2–Coding Challenge
Weightage:	40% of CIA	Total Marks:	100
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	Deva Dharshini	Reg. No.:	192472350
Section / Batch:	BE-CSE-AI,2024	Date:	26-08-2026

Code of Conduct

I Deva Dharshini(192472350) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Deva Dharshini

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Implement Mini-Batch K-Means from scratch to handle a large dataset that doesn't fit efficiently for full-batch updates, and compare clustering quality (inertia, silhouette) and runtime against standard K-Means.

Source Code:

```
import numpy as np

import time

import matplotlib.pyplot as plt

from sklearn.datasets import make_blobs

from sklearn.metrics import silhouette_score

print("Creating dataset...")

X, y = make_blobs(

    n_samples=20000,

    centers=5,

    n_features=2,

    cluster_std=1.2,

    random_state=42

)

print("Dataset created successfully!")

print("Dataset shape:", X.shape)

def standard_kmeans(X, k, max_iters=100):

    n_samples = X.shape[0]
```

```
# Randomly select initial centroids

random_indices = np.random.choice(

    n_samples,

    k,

    replace=False

)

centroids = X[random_indices].astype(float)

for iteration in range(max_iters):

    # Calculate distance from each point

    # to each centroid

    distances = np.linalg.norm(

        X[:, np.newaxis] - centroids,

        axis=2

    )

    # Assign each point to nearest centroid

    labels = np.argmin(

        distances,

        axis=1

    )

    # Store previous centroids

    old_centroids = centroids.copy()
```

```
# Update centroids

for i in range(k):

    cluster_points = X[labels == i]

    if len(cluster_points) > 0:

        centroids[i] = np.mean(

            cluster_points,

            axis=0

        )

# Stop when centroids do not change

if np.allclose(old_centroids, centroids):

    print(

        "Standard K-Means converged at iteration:",

        iteration + 1

    )

    break

return centroids, labels

def minibatch_kmeans(

    X,

    k,

    batch_size=500,

    max_iters=200
```

):

```
n_samples = X.shape[0]
```

```
# Randomly initialize centroids
```

```
random_indices = np.random.choice(
```

```
    n_samples,
```

```
    k,
```

```
    replace=False
```

```
)
```

```
centroids = X[random_indices].astype(float)
```

```
# Number of updates for each centroid
```

```
counts = np.zeros(k)
```

```
for iteration in range(max_iters):
```

```
    # Select random mini-batch
```

```
    batch_indices = np.random.choice(
```

```
        n_samples,
```

```
        batch_size,
```

```
        replace=False
```

```
    )
```

```
    batch = X[batch_indices]
```

```
    # Calculate distance between
```

```
    # batch points and centroids
```

```

distances = np.linalg.norm(
    batch[:, np.newaxis] - centroids,
    axis=2
)

# Assign nearest centroid
labels = np.argmin(
    distances,
    axis=1
)

# Update centroids
for i in range(k):
    cluster_points = batch[labels == i]

    for point in cluster_points:
        counts[i] += 1

        learning_rate = 1 / counts[i]

        centroids[i] = (
            (1 - learning_rate)
            * centroids[i]
            + learning_rate
            * point
        )

```

```

# Final assignment for entire dataset

distances = np.linalg.norm(

    X[:, np.newaxis] - centroids,

    axis=2

)

final_labels = np.argmin(

    distances,

    axis=1

)

return centroids, final_labels

def calculate_inertia(

    X,

    centroids,

    labels

):

    inertia = 0

    for i in range(len(centroids)):

        cluster_points = X[labels == i]

        if len(cluster_points) > 0:

            inertia += np.sum(

                (cluster_points - centroids[i]) ** 2

```

```
    )

    return inertia

print("\nRunning Standard K-Means...")

start_time = time.time()

standard_centroids, standard_labels = standard_kmeans(

    X,

    k=5

)

standard_runtime = time.time() - start_time

standard_inertia = calculate_inertia(

    X,

    standard_centroids,

    standard_labels

)

print("Standard K-Means completed!")

print("\nRunning Mini-Batch K-Means...")

start_time = time.time()

mini_centroids, mini_labels = minibatch_kmeans(

    X,

    k=5,

    batch_size=500
```

```
)

mini_runtime = time.time() - start_time

mini_inertia = calculate_inertia(

    X,

    mini_centroids,

    mini_labels

)

print("Mini-Batch K-Means completed!")

print("\nCalculating Silhouette Scores...")

sample_indices = np.random.choice(

    len(X),

    3000,

    replace=False

)

X_sample = X[sample_indices]

standard_silhouette = silhouette_score(

    X_sample,

    standard_labels[sample_indices]

)

mini_silhouette = silhouette_score(

    X_sample,
```

```
        mini_labels[sample_indices]

    )

    print("\n=====")

    print("    FINAL PERFORMANCE RESULTS")

    print("=====")

    print("\nSTANDARD K-MEANS")

    print(

        "Runtime:",

        round(standard_runtime, 4),

        "seconds"

    )

    print(

        "Inertia:",

        round(standard_inertia, 2)

    )

    print(

        "Silhouette Score:",

        round(standard_silhouette, 4)

    )

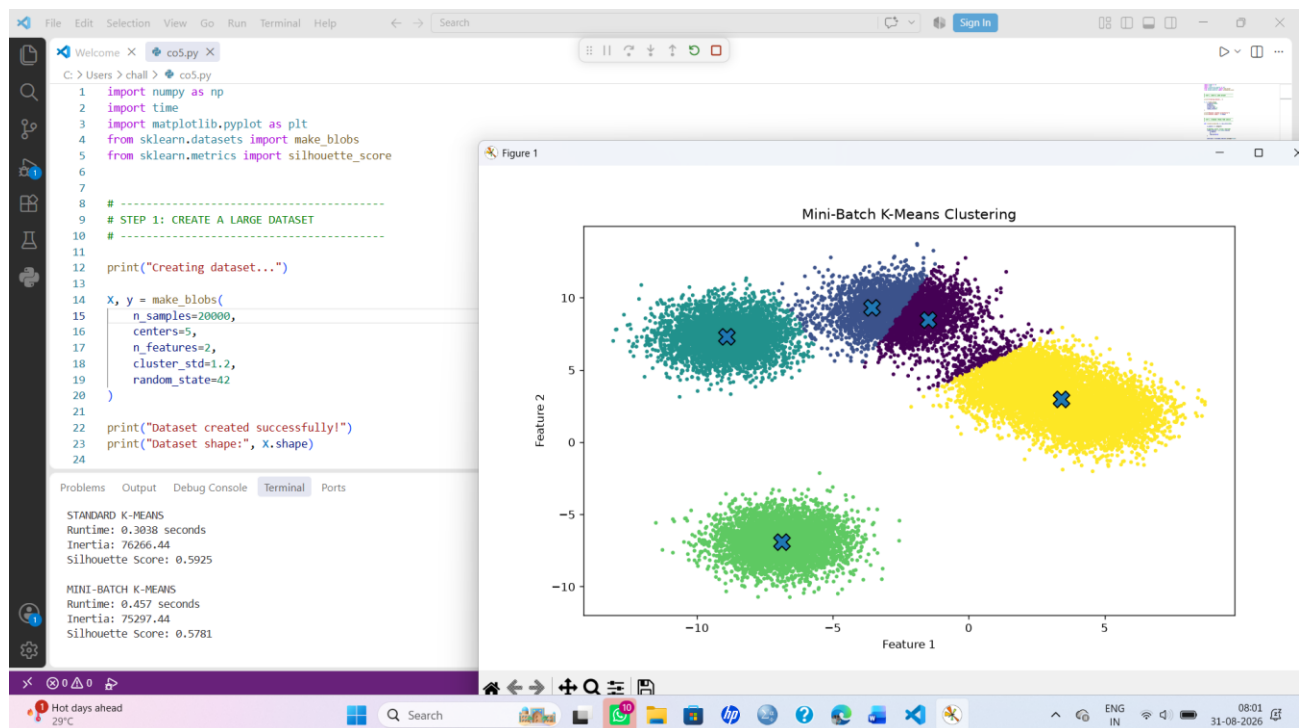
    print("\nMINI-BATCH K-MEANS")

    print(
```

```
"Runtime:",  
  
    round(mini_runtime, 4),  
  
    "seconds"  
  
)  
  
print(  
  
    "Inertia:",  
  
    round(mini_inertia, 2)  
  
)  
  
print(  
  
    "Silhouette Score:",  
  
    round(mini_silhouette, 4)  
  
)  
  
plt.figure(figsize=(10, 6))  
  
plt.scatter(  
  
    X[:, 0],  
  
    X[:, 1],  
  
    c=mini_labels,  
  
    s=5  
  
)  
  
plt.scatter(  
  
    mini_centroids[:, 0],
```

```
mini_centroids[:, 1],  
  
marker="X",  
  
s=200,  
  
edgecolors="black"  
  
)  
  
plt.title("Mini-Batch K-Means Clustering")  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
plt.show()
```

Output:



Geo Tag Photo:

